Architecture

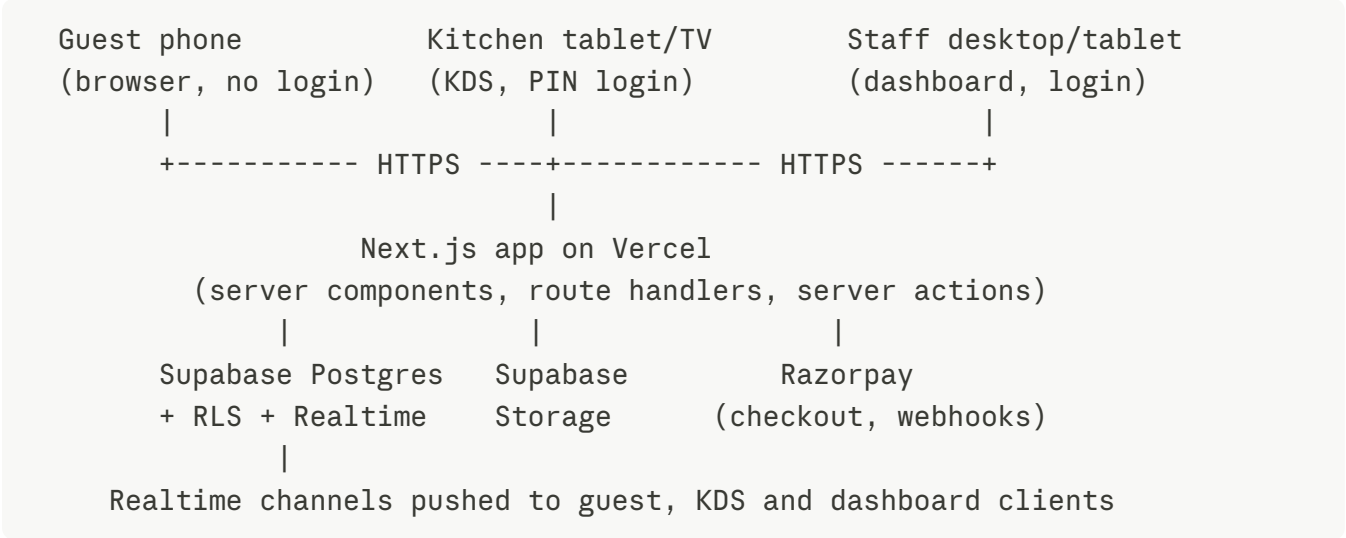
One Next.js application on Vercel backed by Supabase (Postgres, Auth, Realtime, Storage), with Razorpay for payments. Every stack choice here is Inferred: the source page reveals none. The choice favours one codebase, database-enforced tenant isolation, and realtime without running your own socket servers.

1. Stack

Layer	Choice	Why
Language	TypeScript, strict mode	One language end to end
Web framework	Next.js (App Router), React	Server rendering for a fast guest menu; route groups per surface
Styling	Tailwind CSS + shadcn/ui (Radix)	Token-driven, accessible primitives
Client state	Zustand (cart), TanStack Query (server data)	Small bundle; cache and retry built in
Validation	Zod, shared between client and server	One schema per payload
Database	Supabase Postgres with Row Level Security	Tenant isolation enforced in the DB
Auth	Supabase Auth (staff); signed session token (guests)	No guest accounts needed
Realtime	Supabase Realtime (Postgres changes + broadcast)	Order events without socket infrastructure
Storage	Supabase Storage	Menu images, QR PDFs, invoices
Payments	Razorpay Orders + Checkout + Webhooks	UPI and cards in India; hosted card handling
Messaging	SMS/WhatsApp provider behind an interface (e.g. Twilio or MSG91)	Receipts and ready alerts
Hosting	Vercel (app), Supabase (data)	Zero-ops for MVP
Monitoring	Sentry, Vercel Analytics, Supabase logs	Errors, web vitals, DB
Testing	Vitest, Playwright, pgTAP, k6	See Testing Guide

Swap guidance: if you prefer a separate backend, keep the same data model and API contracts and move the route handlers to NestJS or FastAPI on Railway/Render; the rest of this document still holds.

2. System context



3. Application surfaces

One Next.js app, four route groups, each with its own layout and bundle.

Surface	Route prefix	Users	Rendering
Guest	`/t/[qrToken]`, `/o/[orderId]`	Guests	Server-rendered menu, client cart, PWA-friendly
Kitchen	`/kds/[outletId]`	Kitchen	Client-heavy, realtime, wake-lock
Dashboard	`/app/...`	Owner, Manager, Waiter	Server components + client tables
Admin	`/admin/...`	Platform admin	Server components
API	`/api/...`	All	Route handlers

4. Multi-tenancy and access control

- Hierarchy: **tenant** (business) has many **outlets**; staff are **members** of a tenant with a role and optional outlet scope.
- Every business table carries `tenant_id` and `outlet_id`.

- Staff access: RLS policies check `tenant_id` against the caller's memberships (JWT claim or a `members` lookup).
- Guest access: guests never query tables directly. All guest reads and writes go through route handlers using the service role, after validating the QR token and guest session cookie.
- Guest realtime: the server issues a short-lived token allowing subscription to one channel, `order:{orderId}` or `session:{sessionId}`.

Role	Menu	Orders	Payments/refunds	Staff	Settings	Analytics
Owner	Write	Write	Write	Write	Write	Read
Manager	Write	Write	Write	Read	Outlet only	Read
Waiter/Counter	Availability only	Write status, take cash	Record cash	None	None	None
Kitchen	Read	Write status	None	None	None	None

5. Data model

All tables have `id uuid pk`, `created_at`, `updated_at`. Money is stored as integer paise.

Table	Key columns	Notes
`tenants`	name, slug, plan, status, gstin	One per business
`outlets`	tenant_id, name, mode (`dine_in` / `canteen`), address, timezone, open_hours jsonb, tax_config jsonb, service_charge_bps, payment_options jsonb	Mode drives guest flow
`members`	tenant_id, user_id, role, outlet_id nullable, pin_hash, active	Staff
`tables`	outlet_id, label, seats, qr_token (unique, random), status (`free` / `occupied` / `bill_requested`), active	Regenerating `qr_token` invalidates old QR
`menu_categories`	outlet_id, name, sort_order, active	
`menu_items`	outlet_id, category_id, name, description, price, image_path, is_veg, dietary_tags text[], available, sort_order, tax_rate_bps, prep_station	

Table	Key columns	Notes
`item_variants`	item_id, name, price_delta, is_default	Size and similar
`addon_groups`	item_id, name, min_select, max_select	
`addons`	group_id, name, price, available	
`table_sessions`	outlet_id, table_id nullable, token_no nullable, status (`open` / `closed`), opened_at, closed_at, guest_phone nullable	One running bill; canteen sessions have a token and no table
`orders`	outlet_id, session_id, order_no (per-outlet daily sequence), status, note, subtotal, tax, total, placed_at, accepted_at, ready_at, served_at, cancelled_reason, idempotency_key unique	A session can hold many orders
`order_items`	order_id, item_id, name_snapshot, variant_snapshot jsonb, addons_snapshot jsonb, unit_price, qty, note, status	Snapshots keep history stable when the menu changes
`payments`	outlet_id, session_id, method (`upi` / `card` / `cash`), provider, provider_order_id, provider_payment_id, amount, status (`created` / `authorized` / `captured` / `failed` / `refunded`), raw jsonb	
`invoices`	session_id, invoice_no, totals jsonb, pdf_path	GST bill
`order_events`	order_id, type, actor_type, actor_id, payload jsonb	Append-only audit and analytics source
`service_requests`	session_id, type (`call_waiter` / `request_bill`), status	
`inventory_items` , `item_recipes`	Phase 2	Stock and deduction rules
`daily_metrics`	outlet_id, date, orders, revenue, aov, avg_prep_seconds, top_items jsonb	Rollup filled by a scheduled job

Indexes to create up front: `orders(outlet_id, status, placed_at)`, `orders(session_id)`, `order_items(order_id)`, `menu_items(outlet_id, category_id, sort_order)`, `tables(qr_token)`, `payments(provider_order_id)`.

6. Order state machine

The server is the only writer of order status; every transition is validated and logged to `order_events`.

From	To	Who	Condition
(none)	`placed`	Guest	Valid session, items available, totals recomputed server-side
`placed`	`accepted`	Manager/Waiter, or auto	Auto-accept is an outlet setting; canteen requires payment captured first
`placed`	`rejected`	Manager/Waiter	Reason required
`accepted`	`preparing`	Kitchen	
`preparing`	`ready`	Kitchen	
`ready`	`served`	Waiter (dine-in) / Counter (canteen handover)	
`placed`, `accepted`	`cancelled`	Guest (only while `placed`), Manager	Triggers refund if paid

Session lifecycle: `open` on first order, `closed` when the bill is fully paid and staff close the table (dine-in) or when the order is handed over (canteen).

7. API surface

All handlers validate input with Zod, return `{ data }` or `{ error: { code, message } }`, and are rate limited.

Guest (QR token + session cookie)

Method	Path	Purpose
GET	`/api/guest/menu?qr=`	Outlet, table, categories, items; cached and revalidated on menu change
POST	`/api/guest/session`	Open or resume a session for the table; sets an httpOnly cookie
POST	`/api/guest/orders`	Place order; requires `Idempotency-Key` header
GET	`/api/guest/orders/[id]`	Order and status
POST	`/api/guest/orders/[id]/cancel`	Only while `placed`
GET	`/api/guest/session/bill`	Running bill
POST	`/api/guest/payments`	Create a Razorpay order for the amount due
POST	`/api/guest/payments/verify`	Verify checkout signature (webhook remains the source of truth)
POST	`/api/guest/service-requests`	Call waiter, request bill
POST	`/api/guest/realtime-token`	Short-lived token for the guest's channel

Staff (Supabase session, RLS applies)

Method	Path	Purpose
GET	`/api/orders?outlet=&status=`	Live board and KDS initial load
PATCH	`/api/orders/[id]/status`	Transition; body `{ to, reason? }`
PATCH	`/api/order-items/[id]`	Mark an item done
CRUD	`/api/menu/categories`, `/api/menu/items`, `/api/menu/addons`	Menu
PATCH	`/api/menu/items/availability`	Bulk toggle
CRUD	`/api/tables`; POST `/api/tables/[id]/regenerate-qr`; GET `/api/tables/qr.pdf`	Tables and QR
POST	`/api/sessions/[id]/close`	Settle and close; creates invoice

Method	Path	Purpose
POST	`/api/payments/cash` ; POST `/api/payments/[id]/refund`	Cash and refunds
CRUD	`/api/staff`	Members and PINs
GET	`/api/analytics/summary?range=`	Dashboard metrics

Webhooks

Method	Path	Purpose
POST	`/api/webhooks/razorpay`	Verify signature, upsert payment by provider id, update session balance; must be idempotent

8. Realtime design

Channel	Subscribers	Events
`outlet:{id}:orders`	KDS, dashboard	order.created, order.updated, item.updated
`outlet:{id}:tables`	Dashboard	table.updated, service_request.created
`session:{id}`	Guest	order.updated, payment.updated
`outlet:{id}:pickup`	Canteen public display	token.ready, token.collected

Rules: clients load state over HTTP first, then apply events; on reconnect they refetch rather than trust missed events; every event carries `updated_at` so stale events are discarded.

9. Payments

1. Client asks the server to create a payment for the session's amount due. The server computes the amount from the database, never from the client.
2. Server creates a Razorpay order and stores a `payments` row as `created`.
3. Client opens Razorpay Checkout (UPI intent, cards). No card data touches your servers.

4. On success the client posts the signature to `/verify` for a fast UI update.
5. The webhook `payment.captured` is the source of truth and marks the payment `captured`. Handle duplicates and out-of-order delivery by keying on the provider payment id.
6. Refunds are initiated from the dashboard and confirmed by webhook.
7. Cash: staff record a cash payment against the session; requires Waiter role or above.

Multi-restaurant settlement: for a SaaS, use the gateway's marketplace/route product or let each tenant connect its own gateway keys, stored encrypted. Decide before M4.

10. Pricing and tax calculation

A single pure function, `computeBill(lines, outletConfig)`, in a shared package used by the server for truth and by the client for display. Inputs: item price, variant delta, add-ons, quantity, per-item tax rate, service charge. Output: line totals, subtotal, tax breakdown (CGST/SGST), service charge, rounding, total. All in paise, integer maths only.

11. QR codes

- QR encodes `https://<domain>/t/<qr_token>`; the token is a 128-bit random value, not the table id.
- Regenerating a token invalidates printed codes for that table.
- Optional hardening: require the guest to be on-premises (staff approval of first order, or a rotating table PIN) to prevent prank orders from photographed QRs. Auto-accept off is the MVP default safeguard.

12. Security checklist

- RLS enabled on every table; default deny; policies tested with pgTAP.
- Service role key only on the server; never shipped to the client.
- Server recomputes all prices, taxes and totals.
- Idempotency keys on order placement and payment creation.
- Rate limits: order placement per session and per IP; OTP and PIN attempts.
- Webhook signature verification and replay protection.
- PINs hashed (argon2 or bcrypt); KDS devices use a scoped, revocable device session.
- Audit trail in `order_events` for status changes, refunds, cancellations.
- Image uploads: type and size checks, served via storage CDN with transformed sizes.
- Secrets in Vercel/Supabase environment configuration; none in the repo.

13. Performance and caching

- Menu response cached per outlet with tag-based revalidation on any menu write.
- Images: WebP/AVIF, responsive sizes, lazy loaded, blur placeholders.
- Guest route ships no dashboard code; charts and tables are dashboard-only chunks.
- Analytics read from `daily_metrics`, not from raw orders.

14. Environments and deployment

Environment	App	Database	Payments
Local	`next dev`	Supabase CLI local stack	Razorpay test keys
Preview	Vercel preview per PR	Supabase branch or staging project	Test keys
Production	Vercel production	Supabase production, daily backups + PITR	Live keys

Migrations are SQL files in `supabase/migrations`, applied in CI. CI runs lint, typecheck, unit, DB policy tests, then Playwright against the preview.

15. Repository layout

```
apps/web/
  app/(guest)/t/[qrToken]/...      guest menu, cart, tracking
  app/(kds)/kds/[outletId]/...     kitchen screen
  app/(dashboard)/app/...          owner and manager dashboard
  app/(admin)/admin/...            platform admin
  app/api/...                      route handlers
  components/ui/                   design-system primitives
  components/{guest,kds,dashboard}/
  lib/{auth,db,realtime,payments,rate-limit}/
packages/core/                     computeBill, state machine, zod schemas,
types
supabase/migrations/               SQL schema and RLS policies
supabase/tests/                    pgTAP policy tests
tests/e2e/                         Playwright
docs/                              these six documents as markdown
```

16. Phase 2 and 3 extension points

- **Printing:** a small local print agent or cloud-print-capable printers subscribing to `outlet:{id}:orders`.
- **POS integration:** an outbound adapter interface (`pushOrder`, `syncMenu`) with one implementation per vendor, driven by `order_events`.
- **Inventory:** deduct on `accepted` via `item_recipes`; flip `available` at zero.
- **Upsells:** start with rule-based pairs from co-purchase counts in `order_items`; consider a model only once there is data.